

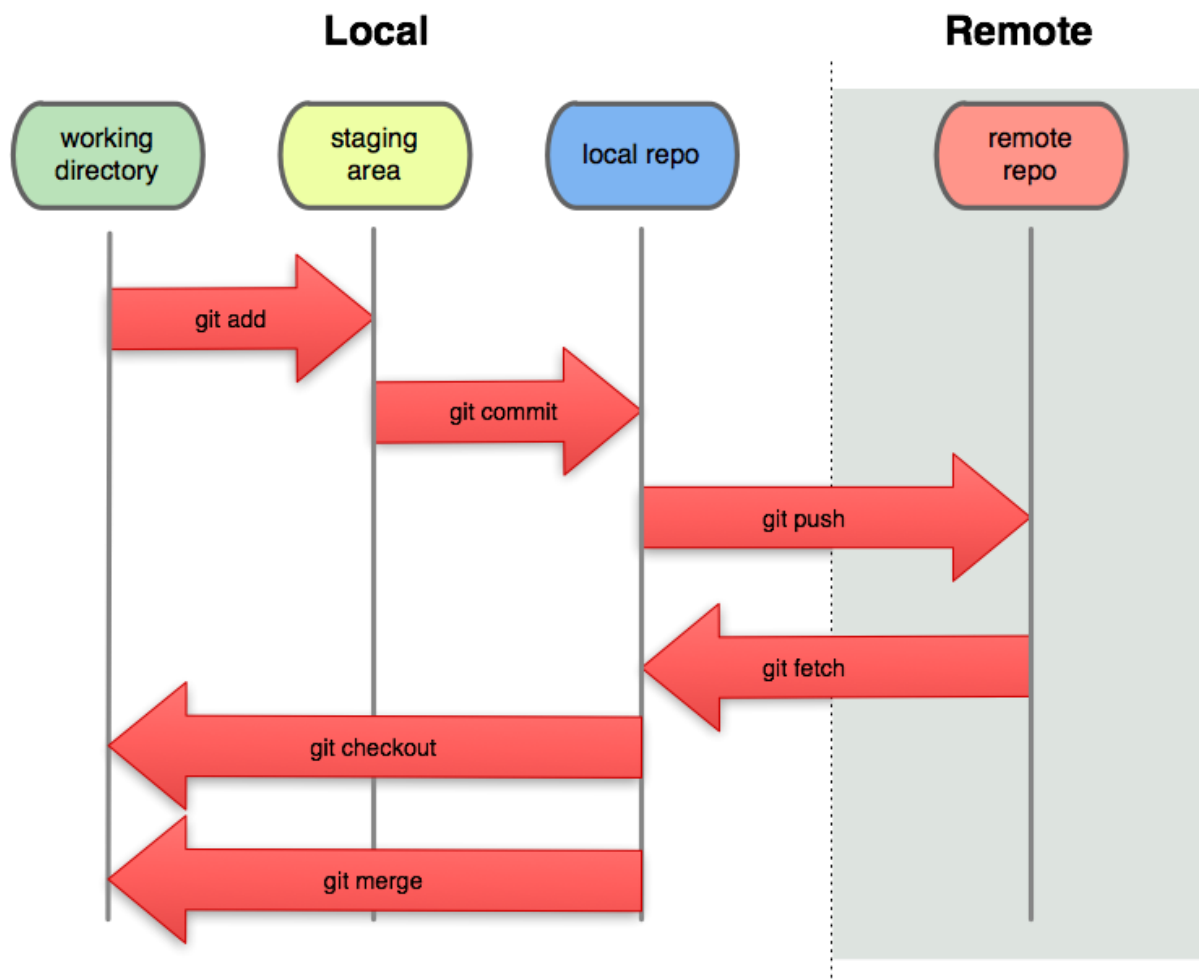
1º DAM - Entornos de Desarrollo

GIT / GitHub

¿Qué es GIT?

GIT es un software de control de versiones, cuyo propósito es llevar registro de los cambios en archivos de computadora y coordinar el trabajo que varias personas realizan sobre archivos compartidos. Existe la posibilidad de trabajar de forma remota y una opción es GitHub.

Flujo de trabajo en GIT



Tenemos nuestro directorio local (una carpeta en nuestro pc) con muchos archivos, Git nos irá registrando los cambios de archivos o códigos cuando nosotros le indiquemos, así podremos viajar en el tiempo retrocediendo cambios o restaurando

versiones de código, ya sea en Local o de forma Remota (servidor externo). En la práctica quedará más claro.

¿Qué es GitHub?

Es una plataforma de desarrollo colaborativo para alojar proyectos (en la nube) utilizando el sistema de control de versiones Git, Además cuenta con una herramienta muy útil que es GitHub Pages donde podemos publicar nuestros proyectos estáticos (HTML, CSS y JS) gratis.

Fundamentos de GIT - Comandos básicos

Los siguientes comandos deberán ejecutarse en consola.

Versión de GIT

git version

Registrar usuario y correo (la primera vez que lo utilicemos)

git config --global user.name "mi nombre"

git config --global user.email "myemail@example.com"

Ayuda

git help

Crear repositorio (solo debo hacerlo una vez por cada proyecto)

git init

Ver estado de los ficheros

git status

git status -s (solo muestra los archivos modificados)

Agregar los archivos que formarán parte del commit (de la fotografía temporal que voy a sacar)

git add nombredelfichero

Crear un commit (fotografía del proyecto en este momento)

git commit -m "primer commit"

Mostrar la lista de commit del más reciente al más antiguo

git log

Trabajando con GIT en Visual Studio Code

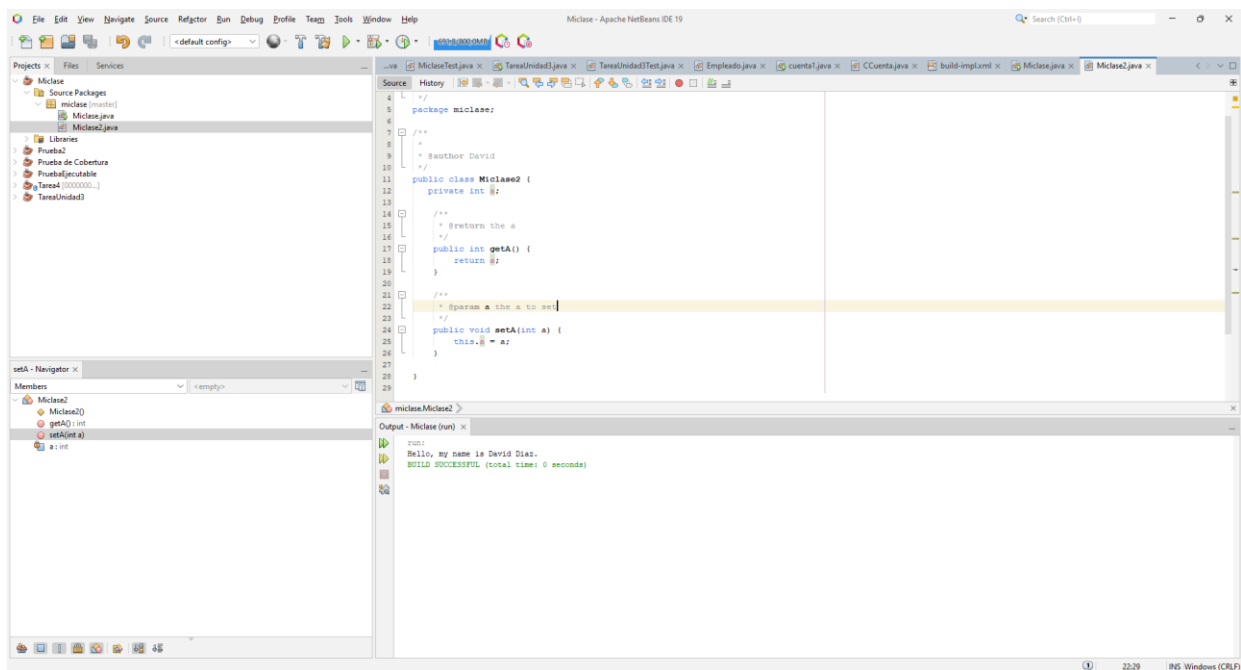
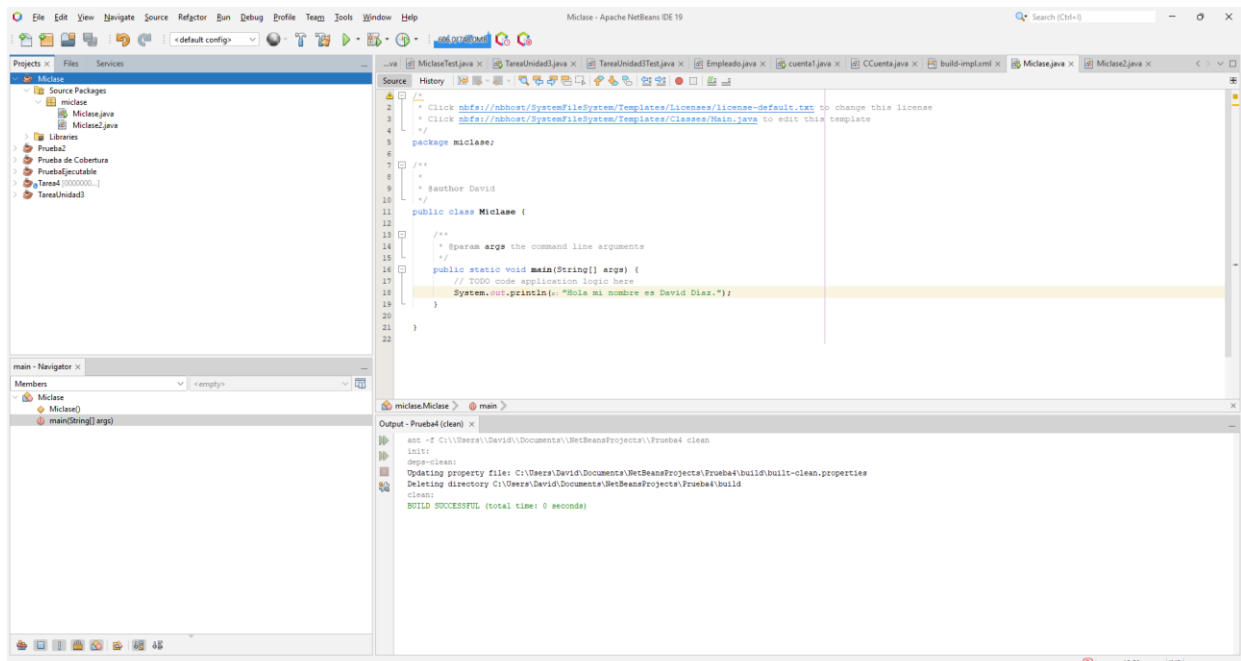
En primer lugar, deberemos descargar GIT del siguiente

enlace. <https://git-scm.com/>

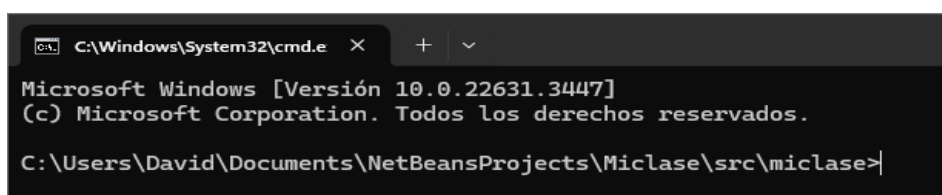


Una vez descargado simplemente instalaremos GIT en nuestro equipo siguiendo los pasos del instalador. Siempre pulsando siguiente.

A continuación, crearemos un proyecto del IDE Net Beans con dos clases `Miclase` y `Miclase2`.



Y abrimos una terminal con el comando “cmd” en el explorador de archivos, deberemos ahora movernos al directorio donde está nuestros ficheros Java.



Comprobamos que tenemos GIT instalado correctamente.

```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git version
git version 2.45.0.windows.1
```

Nota: Es posible que la primera vez que trabajemos con GIT veamos un error en el que se nos solicita un nombre de usuario y un correo, en ese caso solo deberemos ejecutar las siguientes instrucciones.

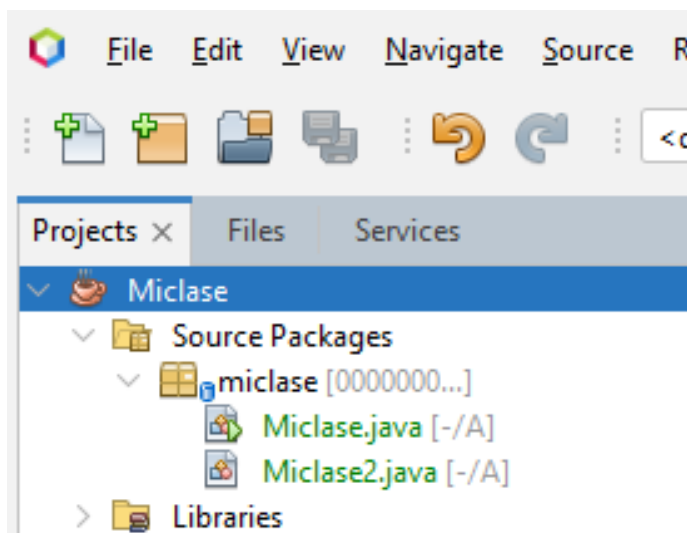
```
git config --global user.name "mi nombre"
```

```
git config --global user.email "myemail@example.com"
```

Ahora creamos el repositorio de este proyecto.

```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git init
Initialized empty Git repository in C:/Users/David/Documents/NetBeansProjects/Miclase/src/miclase/.git/
```

Vemos que ambas clases me las muestra como “sin seguimiento”.

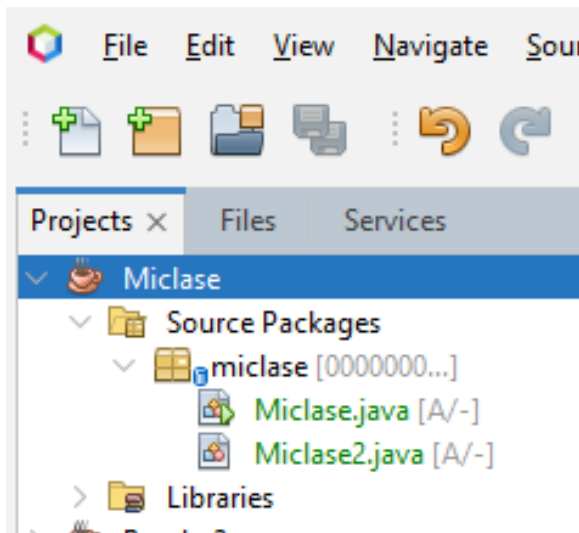


Un git status -s me mostrará que ambos ficheros aún no tienen un seguimiento realizado en GIT. Aparece en el IDE con el carácter **A** al final.

```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git status -s
?? Miclase.java
?? Miclase2.java
```

Añado los dos ficheros a la zona temporal previa a sacar la “fotografía temporal” con **add** y vemos que cambia la letra que nos aparece al lado de ambos ficheros.

```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git add Miclase.java
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git add Miclase2.java
```



Aparece en el IDE con el carácter **A** al principio.

```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git status -s
A Miclase.java
A Miclase2.java
```

Nota: En caso de obtener este Warning.

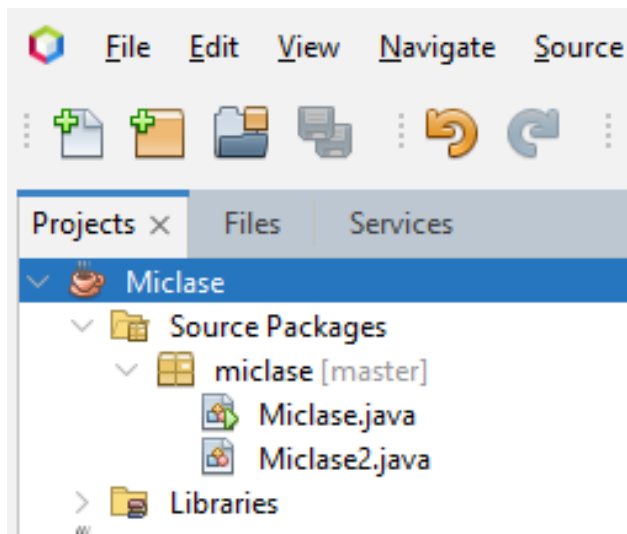
```
warning: in the working copy of 'Miclase.java', LF will be replaced by CRLF the next time Git touches it
```

Podremos eliminarlo con esta instrucción.

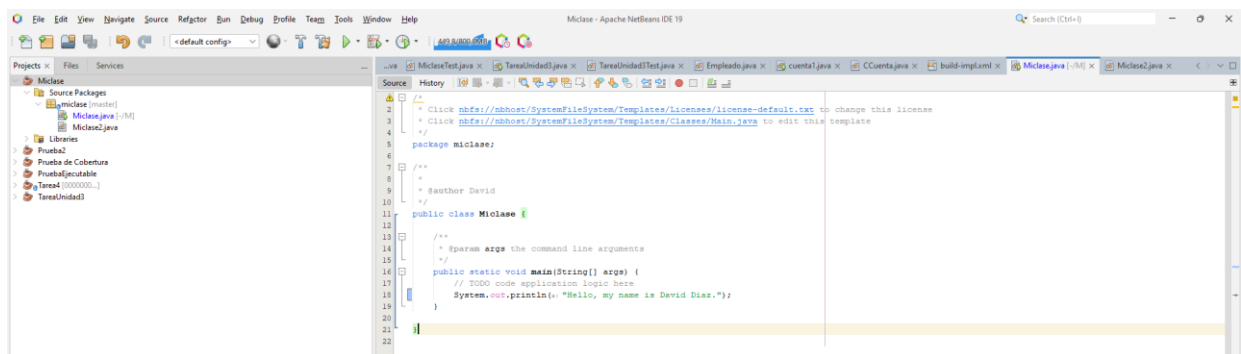
```
git config --global core.autocrlf false
```

Hacemos el commit y le añadimos un mensaje para saber en qué momento hicimos esa “fotografía” del proyecto.

```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git commit -m "Añadimos la primera foto con las dos clases Java"
[master (root-commit) 01a7982] Añadimos la primera foto con las dos clases Java
 2 files changed, 49 insertions(+)
 create mode 100644 Miclase.java
 create mode 100644 Miclase2.java
```



Vamos ahora a cambiar el mensaje de la clase Miclase de inglés a español.



Y como hemos realizado cambio en el fichero “Miclase.java”, aparece el carácter **M** para indicarme que ha sido modificado y que no hemos hecho commit de la última versión.



Hacemos los add y el commit correspondiente.

```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git add Miclase.java
```

```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git commit -m "Añadimos la segunda foto con las dos clases Java"
[master 625b88a] Añadimos la segunda foto con las dos clases Java
1 file changed, 1 insertion(+), 1 deletion(-)
```

Y vemos que la clase ya no tiene el carácter **M**.



Si hacemos un git log me mostrará el registro de commit realizados.

```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git log
commit 625b88aa2622f01971b95328728edeceb2a10a43 (HEAD -> master)
Author: David Diaz <josedaviddiazp@gmail.com>
Date: Sun May 5 20:47:25 2024 +0100

    Añadimos la segunda foto con las dos clases Java

commit 01a79824cb9a49f3f47102b3805697e0a8d7e8ce
Author: David Diaz <josedaviddiazp@gmail.com>
Date: Sun May 5 20:40:55 2024 +0100

    Añadimos la primera foto con las dos clases Java
```


Pasando nuestro repositorio a GitHub

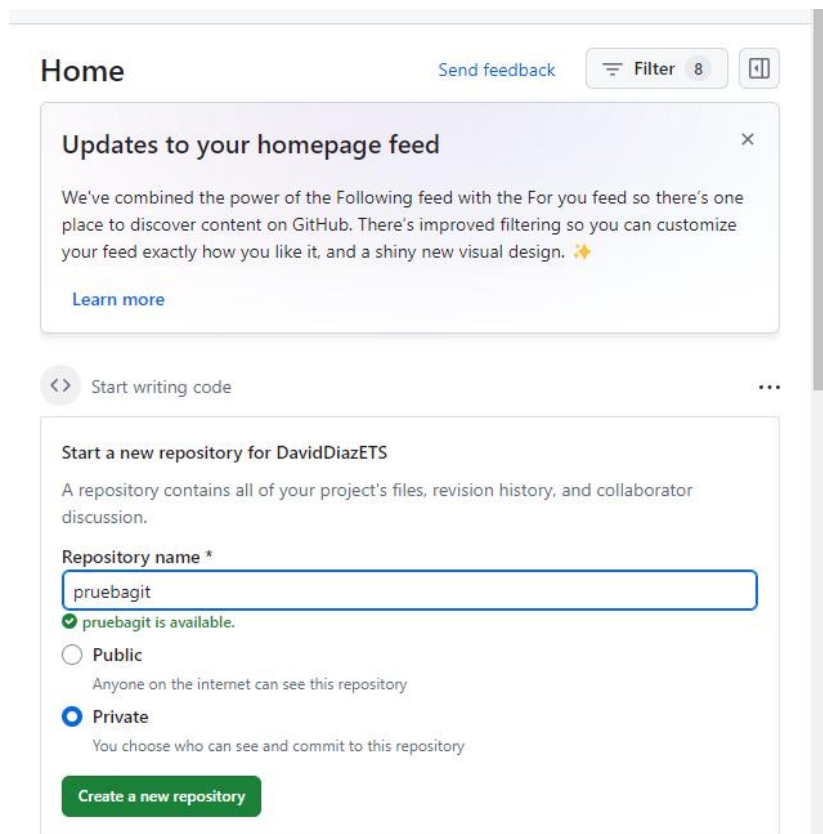
A continuación, lo que vamos a hacer es enviar el repositorio con toda la información de nuestro proyecto y de las “fotografías temporales” sacadas mediante commit a la plataforma GitHub para que sean almacenado en la nube.

Para ello debemos darnos de alta en GitHub en la siguiente página.

<https://github.com/>

Una vez registrados crearemos nuestro repositorio online donde volcaremos el contenido del repositorio que tenemos en nuestro equipo.

Damos un nombre a nuestro nuevo repositorio el cual ya nos aparece asignado como propietarios.

The image shows a screenshot of the GitHub web interface. At the top, there's a 'Home' header with a 'Send feedback' link, a 'Filter 8' button, and a grid icon. Below this is a notification box titled 'Updates to your homepage feed' with a close button (X) and a 'Learn more' link. Underneath is a button that says '<> Start writing code'. The main section is titled 'Start a new repository for DavidDiazETS'. It contains a description: 'A repository contains all of your project's files, revision history, and collaborator discussion.' Below this is a text input field for 'Repository name *' with the value 'pruebagit'. A green checkmark icon and text 'pruebagit is available.' are shown below the input field. There are two radio button options: 'Public' (unselected) with the description 'Anyone on the internet can see this repository' and 'Private' (selected) with the description 'You choose who can see and commit to this repository'. At the bottom is a green button labeled 'Create a new repository'.

Y también podremos establecer si queremos que ese repositorio sea público (cualquier usuario podrá verlo, y podremos decidir quién queremos que haga commit en nuestro proyecto) o privado (nosotros escogemos quien puede verlo y hacer commit).

Y pulsamos **create repository**.

Una vez creado nos llevará a una pantalla donde nos mostrará las dos siguientes líneas que debemos ejecutar en la terminal para indicarle a nuestro proyecto que ahora va a trabajar en remoto con un repositorio en la nube.

```
git remote add origin https://github.com/DavidDiazETS/pruebagit.git
```

```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git remote add origin https://github.com/DavidDiazETS/pruebagit.git
```

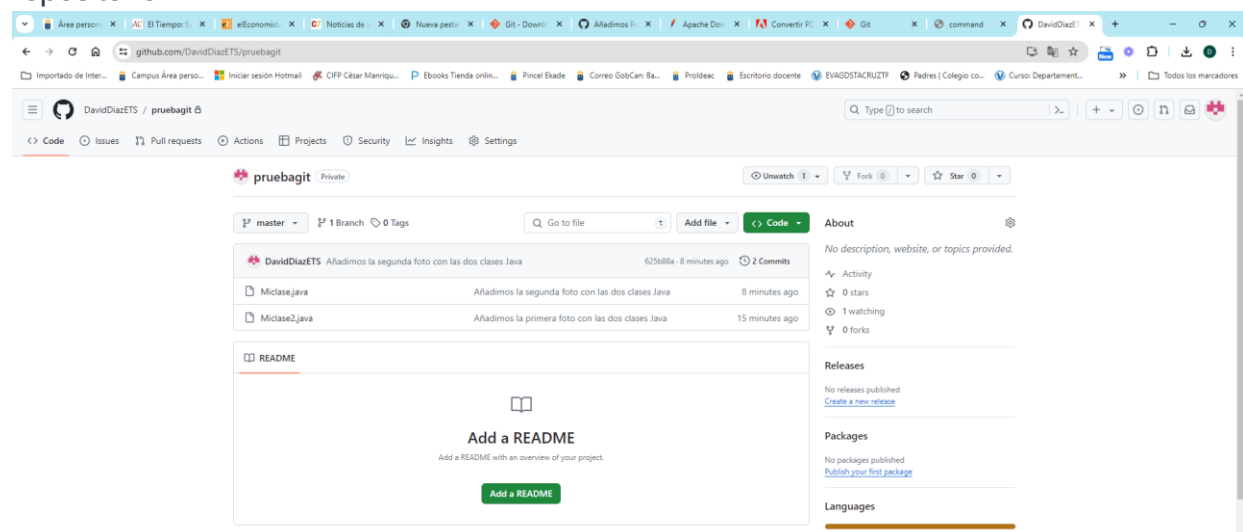
Y la instrucción para enviar nuestros commit al repositorio en la nube. Es posible que la primera vez que hagamos el push nos pida introducir nuestras credenciales de GitHub.

```
git push -u origin master
```

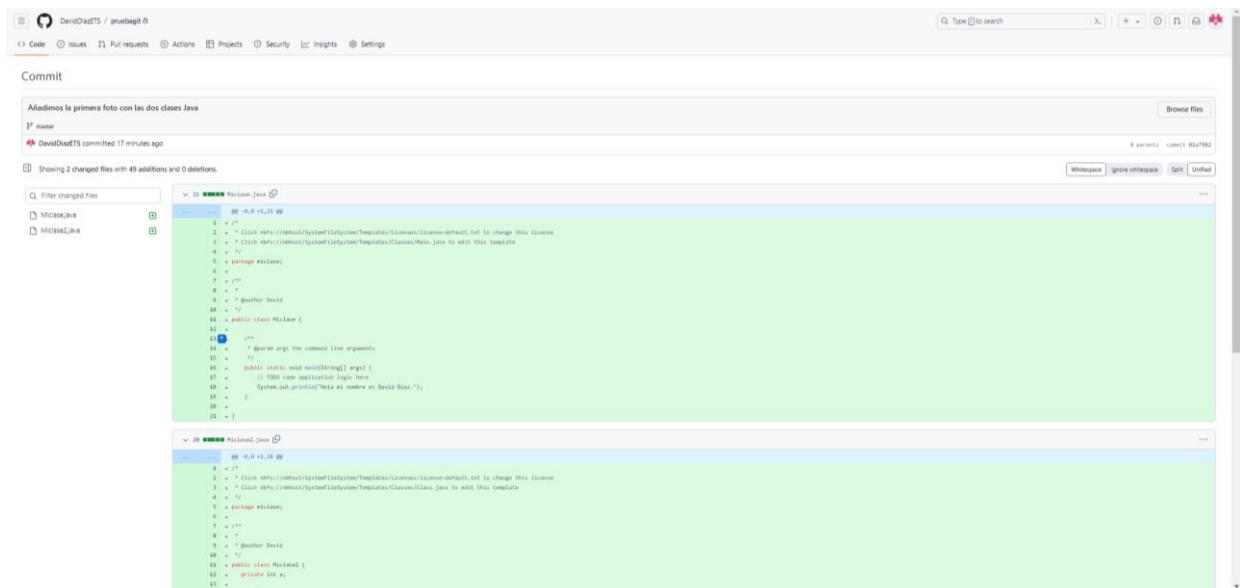
```
C:\Users\David\Documents\NetBeansProjects\Miclase\src\miclase>git push -u origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.12 KiB | 1.12 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/DavidDiazETS/pruebagit.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Nota: Hay que fijarse en que en la instrucción generada no haya puesto main en lugar de master. Si ejecutamos con main no funcionará. Por lo que cambiaremos main por master en la instrucción.

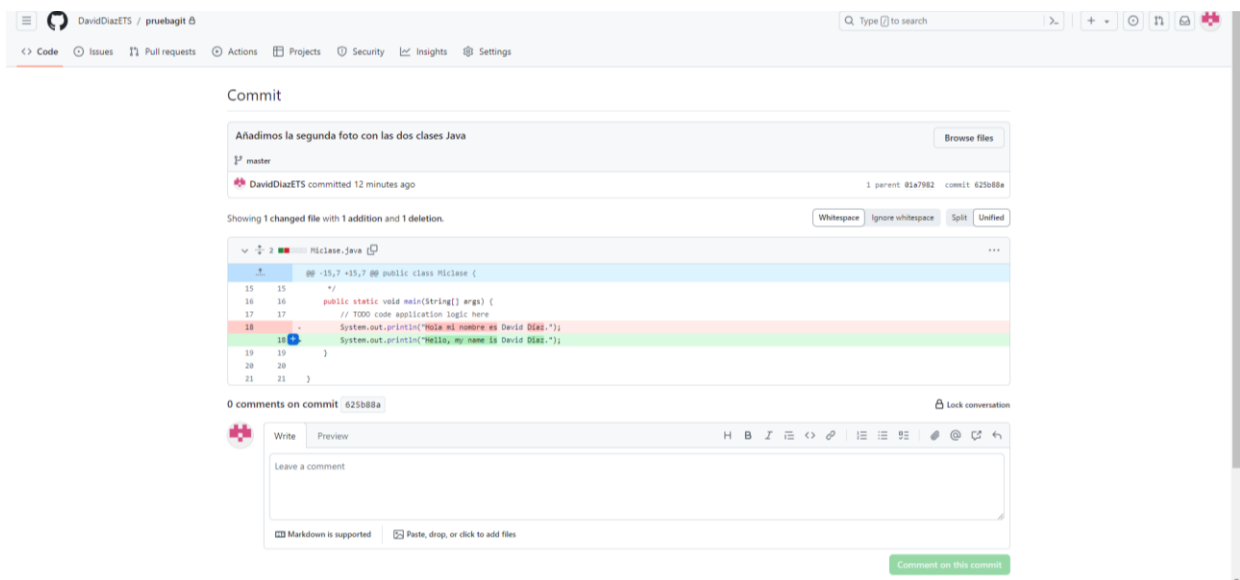
Si volvemos a GitHub vemos que ya tenemos la información volcada en el repositorio.



Nuestro primer commit.



Y el segundo commit con el cambio indicado.



Y ahora ya podremos seguir haciendo cambios en cualquiera de los ficheros de nuestro proyecto. Simplemente tendremos que recordar la secuencia de instrucciones a realizar.

- git add con los ficheros que hayamos modificado (no es necesario hacer un add de todos los ficheros del proyecto para cada “fotografía”, solo de aquellos que hayamos modificado).
- git commit -m “mensaje” para sacar la “fotografía temporal” con un mensaje explicativo.
- git push para enviar el commit a nuestro repositorio en la nube en GitHub.

Ejercicio de práctica

Crea un proyecto Java en el IDE NetBeans que tenga 3 clases diferentes en su interior.

- Realiza todos los pasos necesarios en GIT para tener el repositorio local y en la nube de tu proyecto.
- A continuación, añade algún fragmento de código en cada uno de las tres clases y envía toda la información al repositorio
- Haz cambios en los códigos de la clase 1 y la clase 2 y envía los cambios al repositorio.
- Haz cambios en los códigos de la clase 2 y y la clase 3 y envía los cambios al repositorio.
- Vuelve a hacer cambios en el código de la clase 2 y envía los cambios al repositorio.
- Comprueba en GitHub que todos los commit se realizaron correctamente y que tienes todas las diferentes versiones de tu código almacenadas en la nube de manera correcta.